

Distributed Data Processing Environments

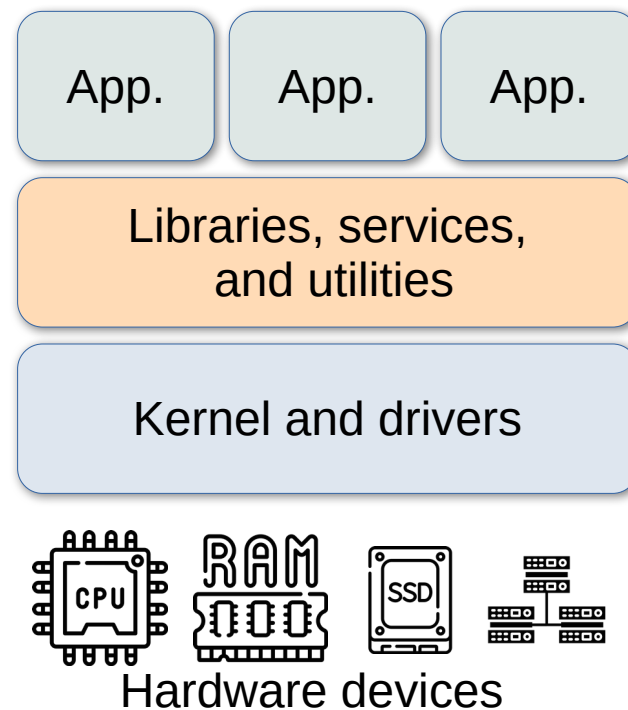
José Orlando Pereira

Departamento de Informática
Universidade do Minho



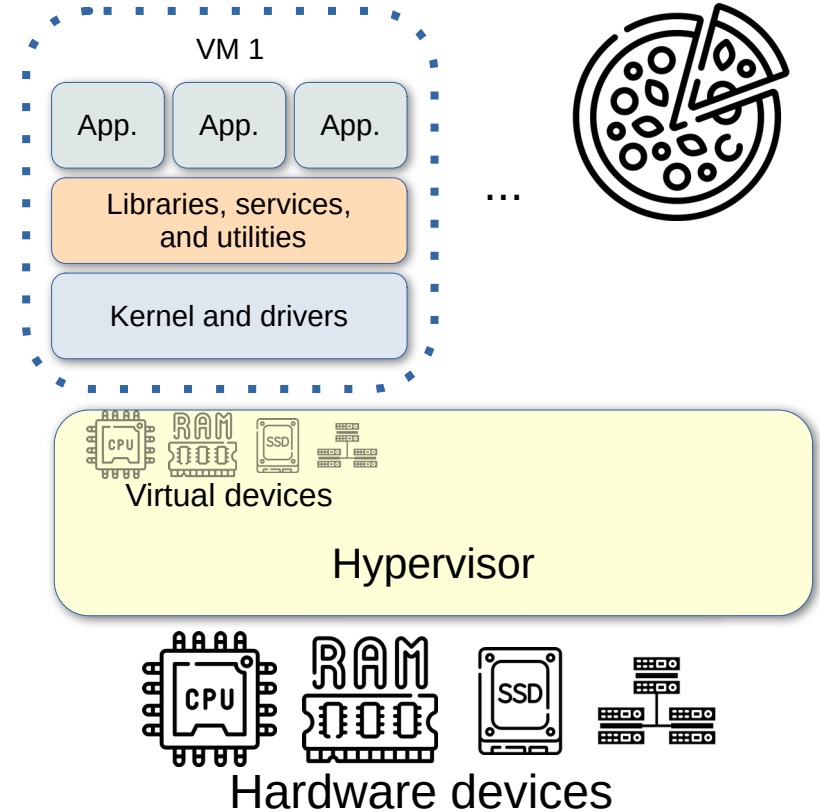
Operating system stack

- Libraries, services, and utilities
 - e.g., user interface, ...
- Kernel and device drivers
 - Encapsulates hardware
 - Protects resources
 - e.g., scheduler, file systems, ...



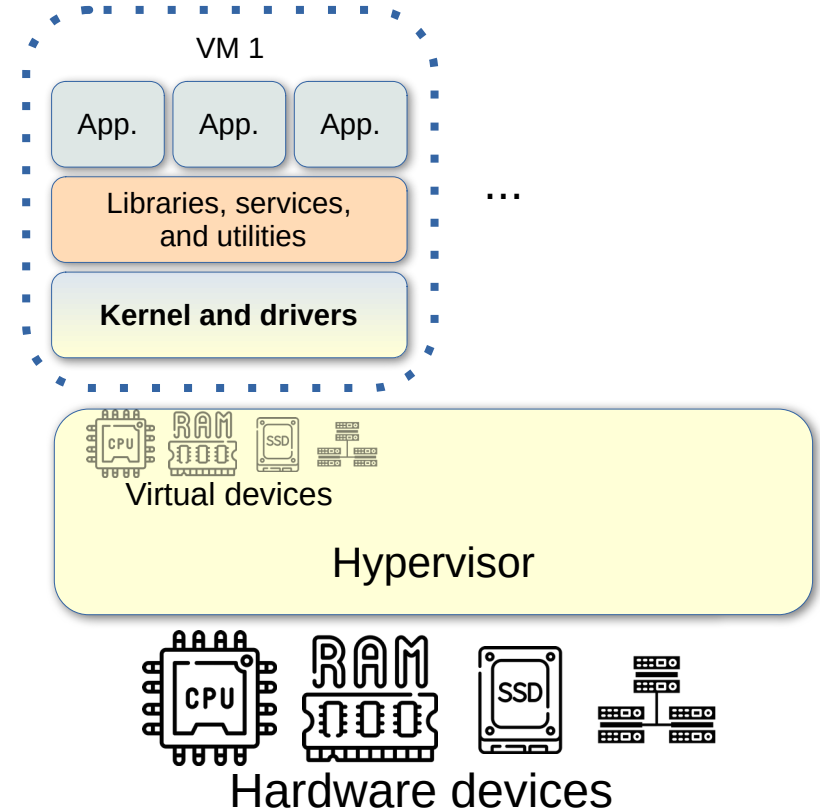
Virtualization

- Slice hardware resources for different users / applications
- Each slice looks like an actual machine
 - Virtual machine
- Isolate slices from each other:
 - Security
 - Performance



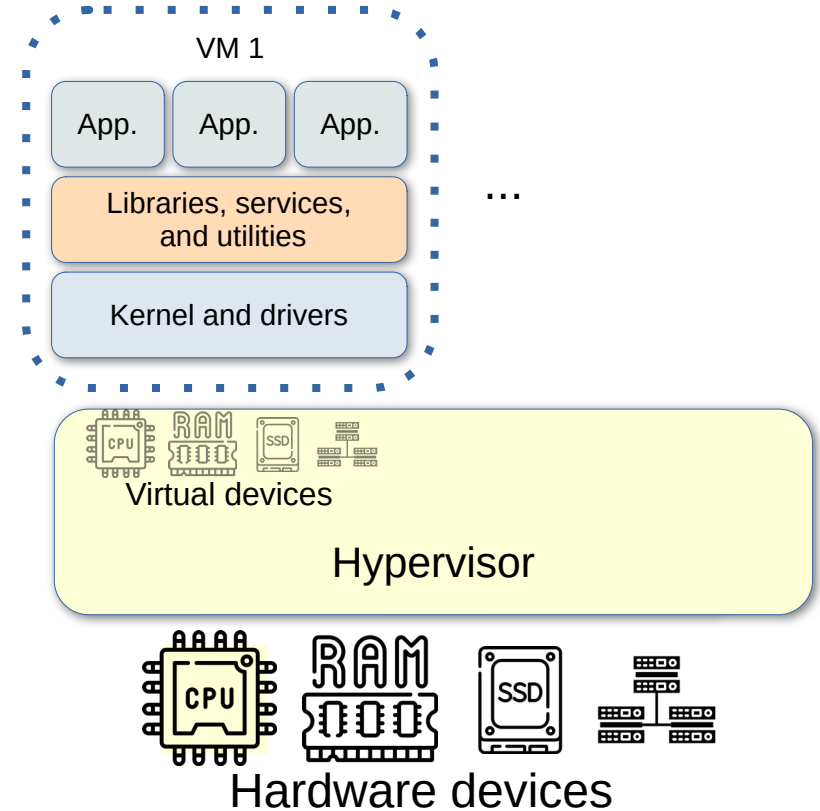
Paravirtualization

- How to trick the kernel into accepting virtual devices?
- Modify kernel and/or device driver code to directly use hypervisor services



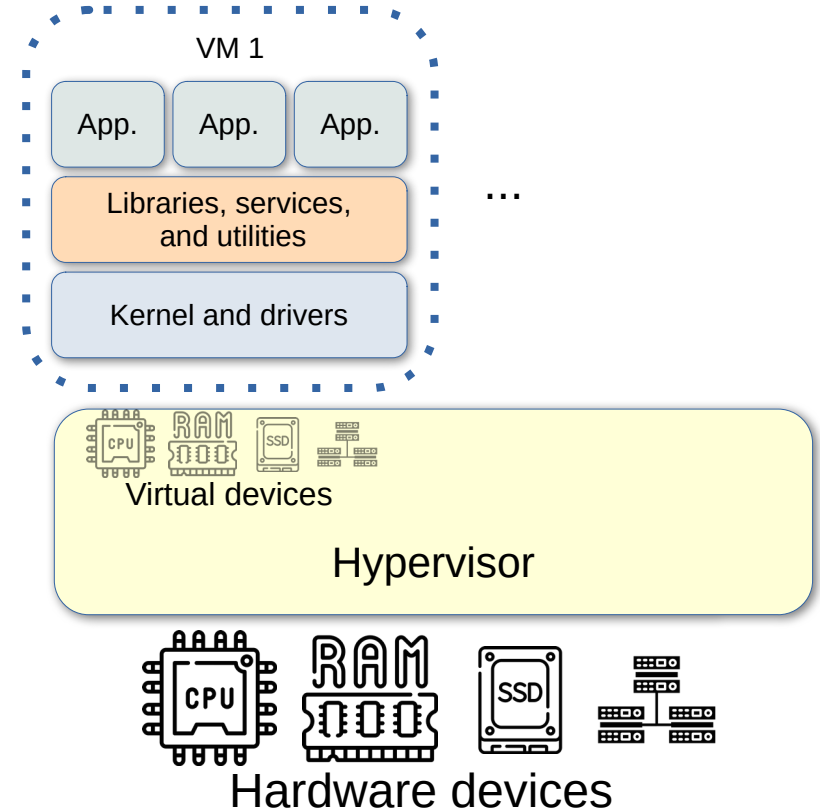
Full virtualization

- How to trick the kernel into accepting virtual devices?
- Modify CPU to route VM operations to hypervisor services
 - Privileged operations



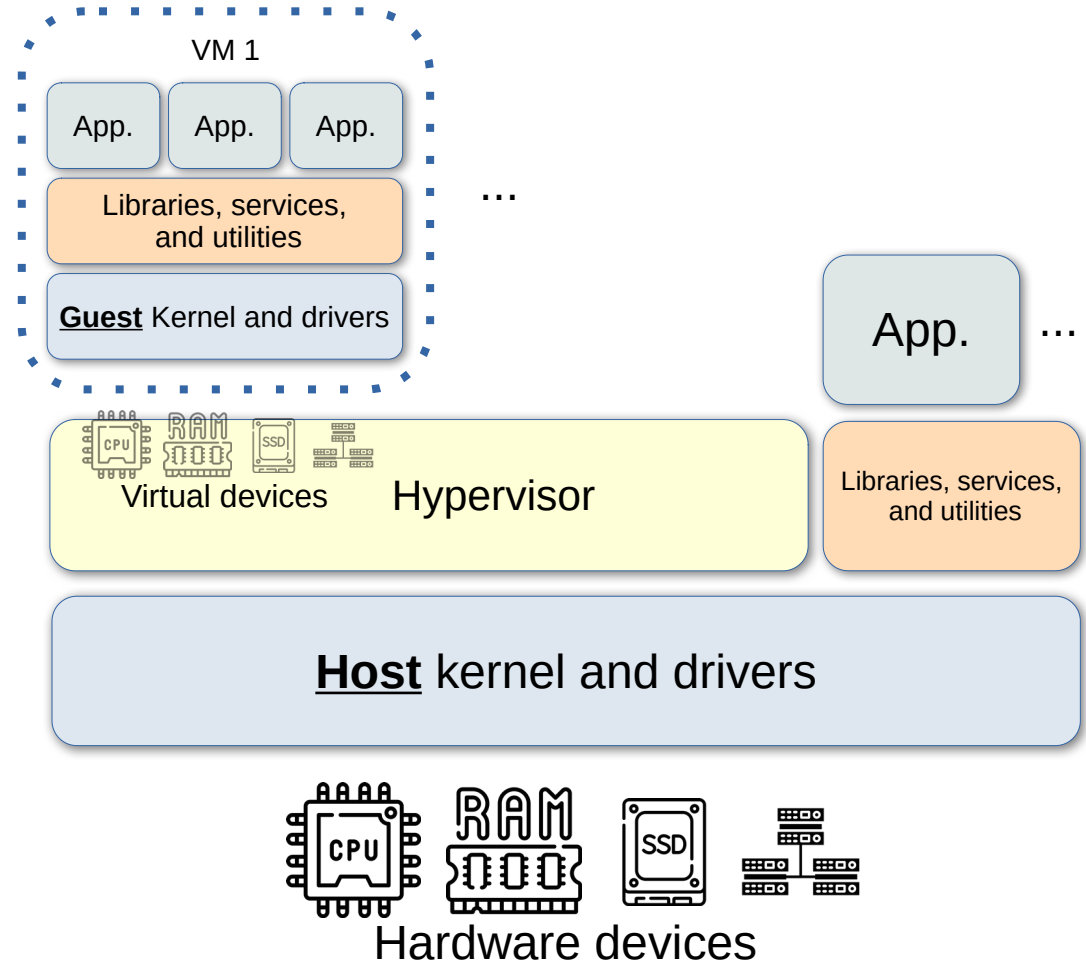
Bare metal

- How to bootstrap the hypervisor?
- The hypervisor is itself a small operating system kernel
 - Custom device drivers



Hosted

- How to bootstrap the hypervisor?
- The hypervisor runs on top of a normal operating system kernel
 - Host kernel provides support



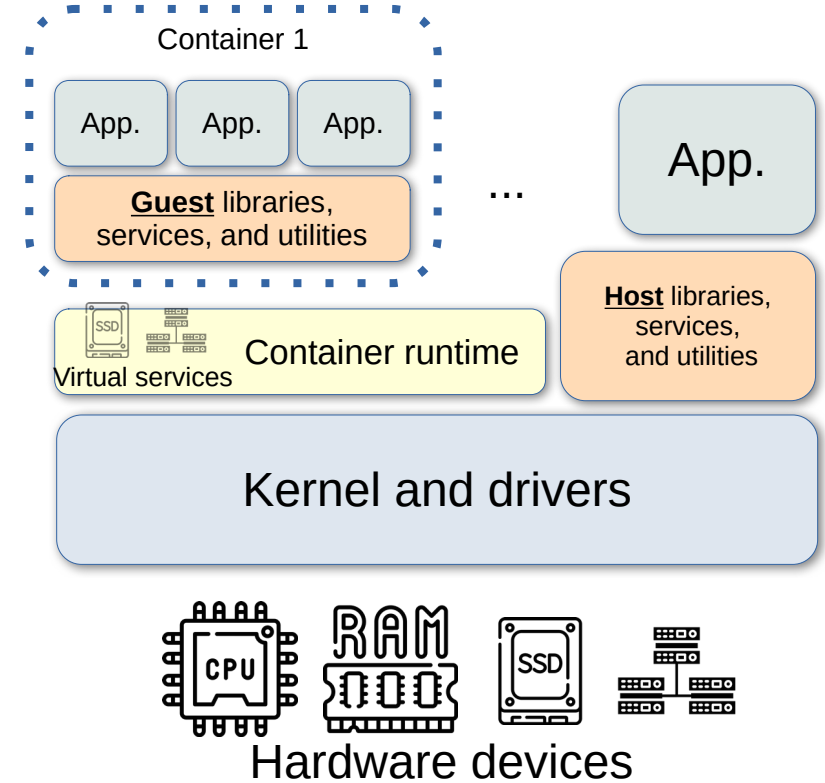
Examples

- Full / bare metal
 - Xen (modern), VMWare ESX
- Paravirtualization / bare metal
 - Xen (original)
- Full / hosted
 - Linux KVM, VirtualBox
- Paravirtualization / hosted
 - VirtualBox with “Guest Additions”



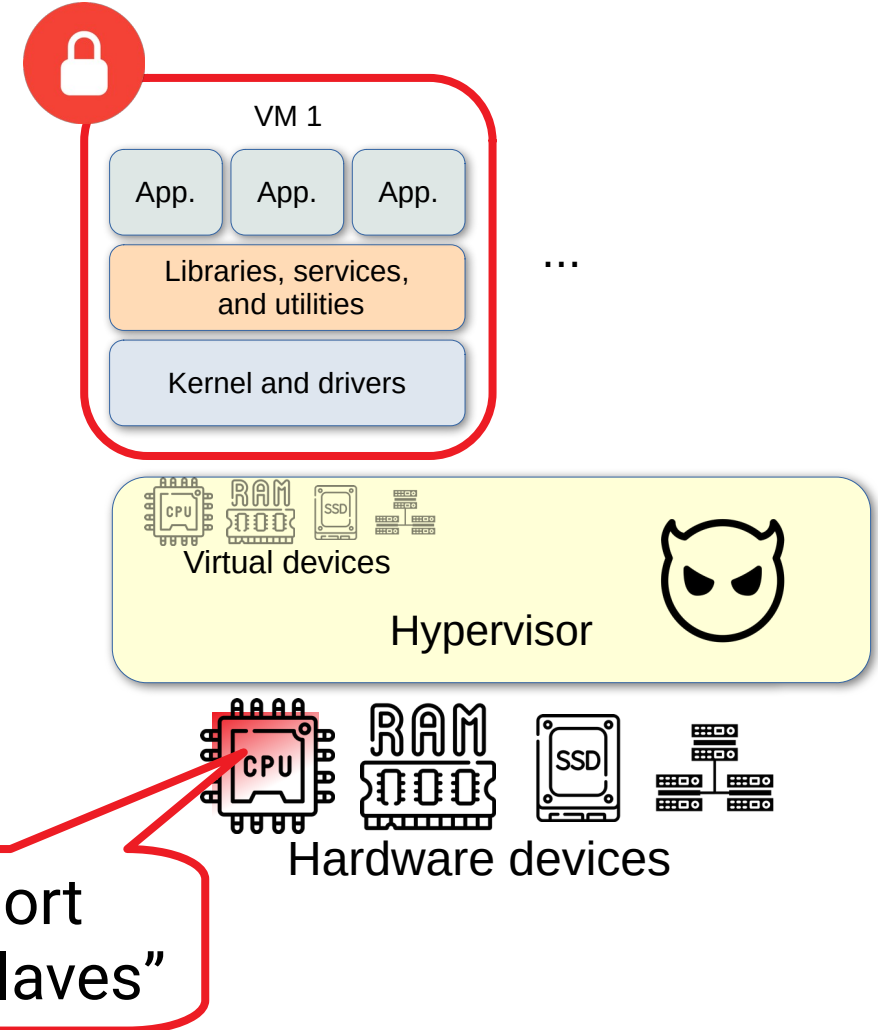
Containerization

- Weaker isolation
- Lower performance overhead
 - Faster setup and tear down
- Examples:
 - Docker / Podman
 - Kubernetes (K8s)



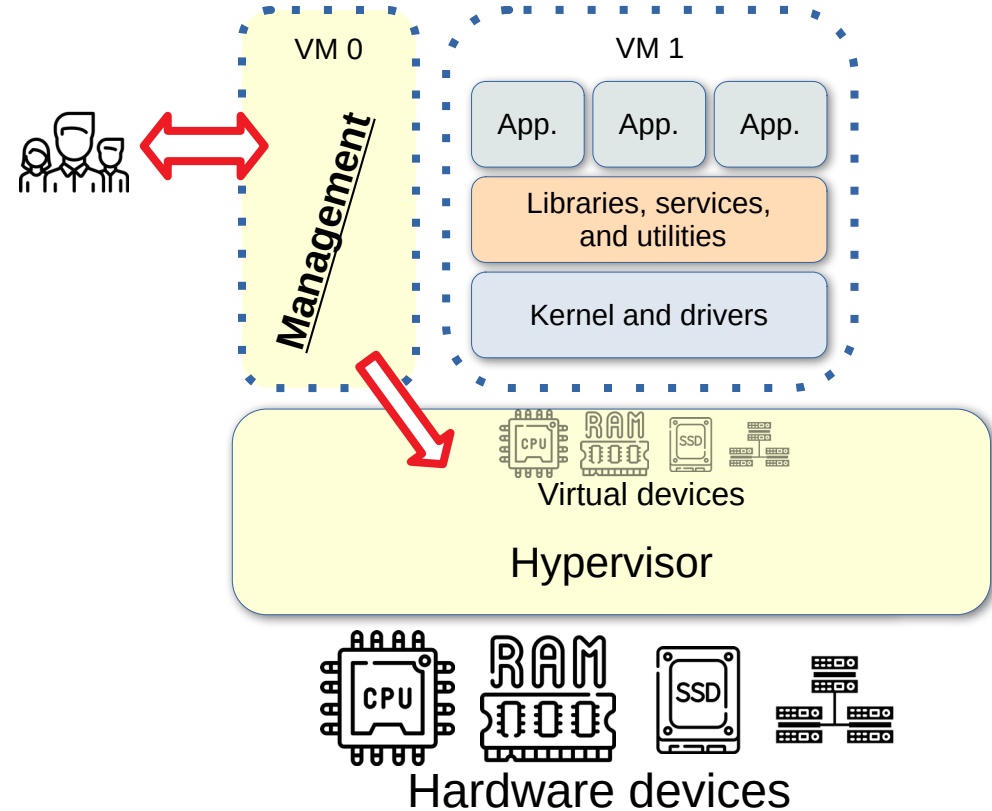
Trusted execution

- Isolation from a malicious hypervisor
 - Data is signed and encrypted
- Relies on the CPU to enforce isolation
- Examples:
 - Intel SGX, ARM TrustZone



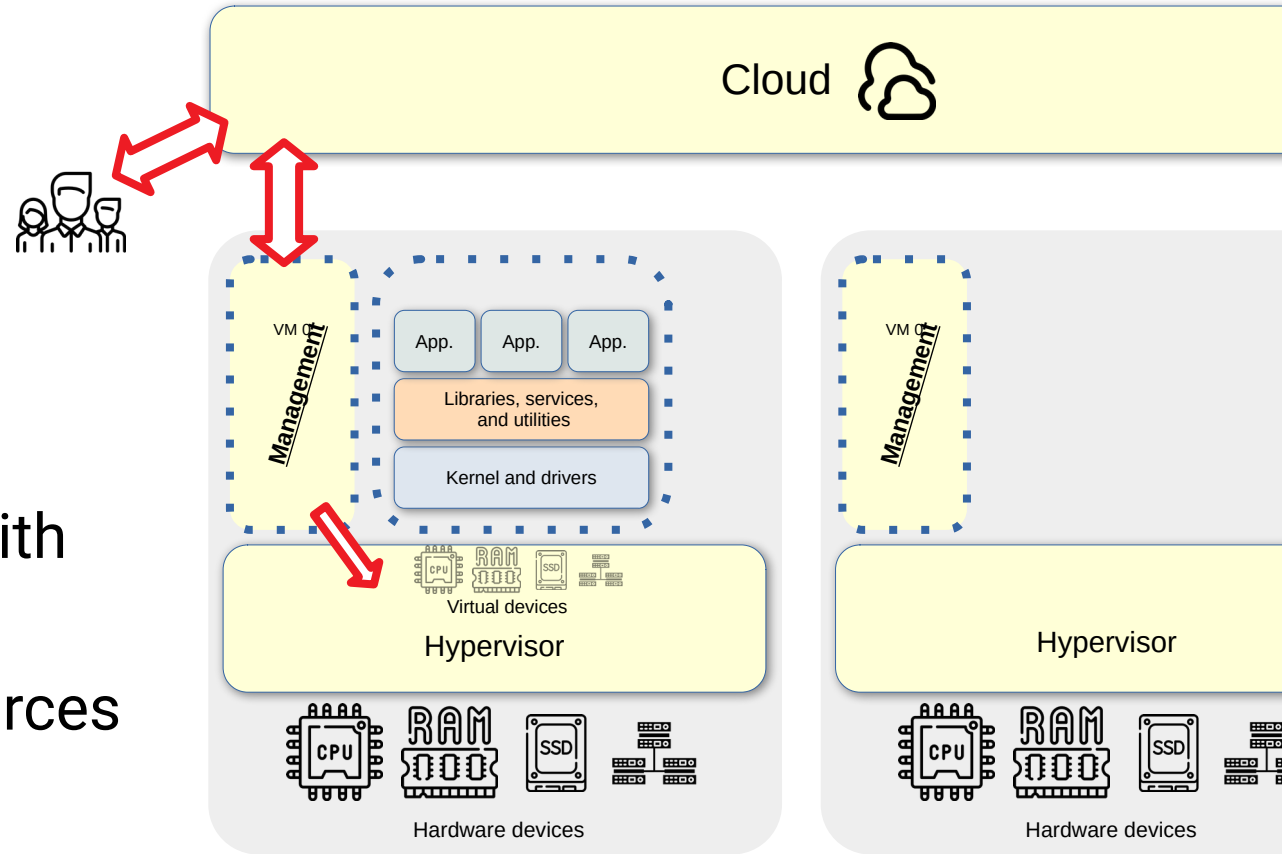
Remote management

- Add a management service to the physical host:
 - As an additional VM
 - As a host process
- The management service allows
 - Creating, starting, stopping, destroying VMs
 - Allocating physical to virtual resources



Cloud services

- Do not directly contact hosts
- A central service:
 - Routes provisioning requests to hosts with available resources
 - Bills users for resources used



Elasticity

- The ability to dynamically add and remove capacity according to actual needs
 - Avoids expensive over-provisioning
- Elasticity can be managed:
 - Cloud services monitor usage of allocated resources and workload
 - More resources are automatically added when needed
 - Resources are automatically removed when idle

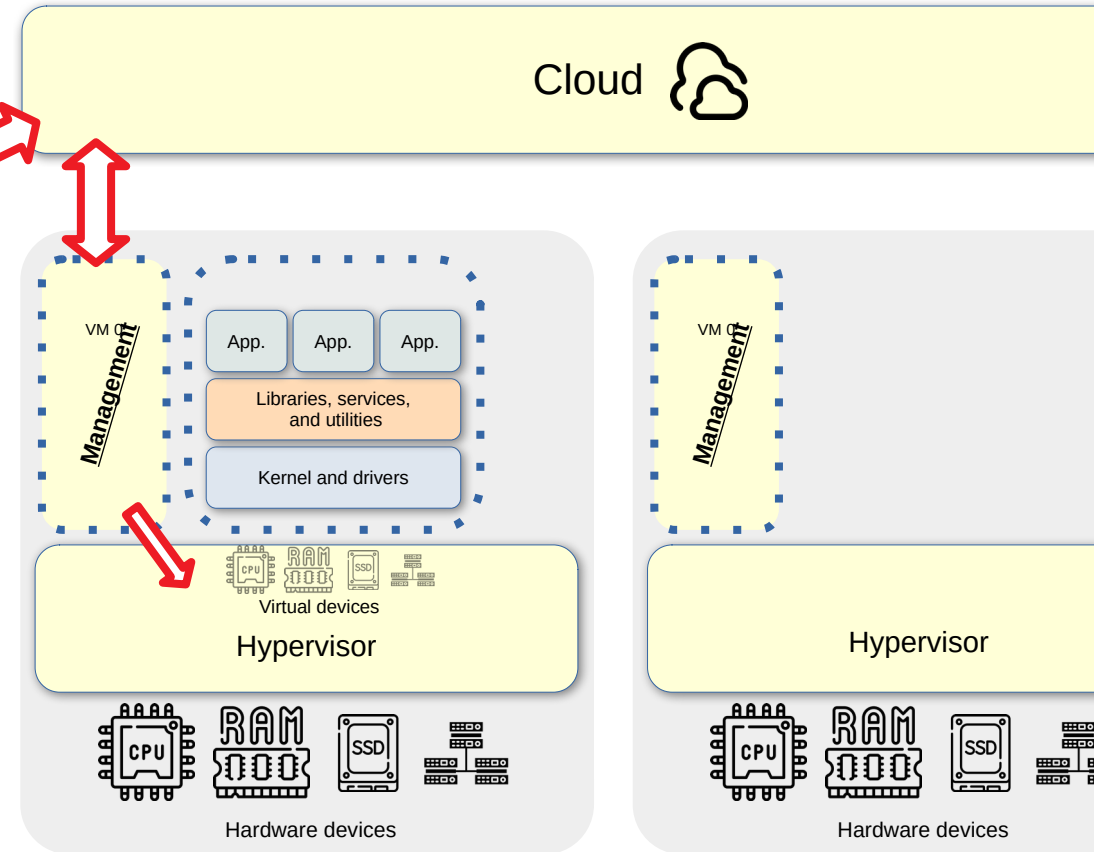
Cloud services

- **Infrastructure as a Service (IaaS)**
 - Raw resources provided by hypervisors
 - Optional managed elasticity (e.g., with K8s)
- **Platform as a Service (PaaS)**
 - Services used by application developers
 - Managed elasticity
 - Examples: Storage (S3), DBaaS (Aurora), FaaS
- **Software as a Service (SaaS)**
 - Services for end-users
 - Managed elasticity
 - Examples: GMail, ...

Infrastructure as Code (IaC)

- The user does not directly interact with provisioning
- Instead, they write programs that control provisioning
- Provisioning code can be managed as usual (git, ...)

```
5  mov     eax, DWORD PTR [rbp-20]
6  imul    eax, eax
7  mov     DWORD PTR [rbp-8], eax
8  mov     eax, DWORD PTR [rbp-8]
9  add     eax, eax
10 mov     DWORD PTR [rbp-12], eax
11 mov     DWORD PTR [rbp-4], 5
12 mov     eax, DWORD PTR [rbp-4]
13 sub     eax, DWORD PTR [rbp-12]
14 test    eax, eax
15 jns     .L2
16 mov     eax, DWORD PTR [rbp-12]
17 add     DWORD PTR [rbp-4], eax
18
.L2:
19 mov     edx, DWORD PTR [rbp-4]
20 mov     eax, DWORD PTR [rbp-8]
21 add     edx, eax
22 mov     DWORD PTR [rbp-12], edx
```



Provisioning

- Step 1: Select VMs / containers and hardware resources
 - CPU, RAM, storage, networking
- Step 2: Install and configure software
 - Operating system base
 - Libraries and services
 - User application
 - Configuration parameters

Key Issue: Security and access!

Provisioning styles

- Imperative: **How** to assemble needed infrastructure?
 - The user describes the steps to perform
 - The user validates that the system is in the correct state and reruns from scratch when needed
- Declarative: **What** infrastructure is needed?
 - The system decides the steps to perform
 - From scratch or from an existing running infrastructure
 - Validates that the system is in the correct state and takes corrective action
 - On demand, or continuously

Provisioning scope

- Single instance
 - Describes a single instance
 - Focuses on software installation and configuration
- Orchestration
 - Describes multiple instances
 - Focuses on relation between instances
 - Networking resources
 - Multi-instance constraints (e.g., “at least 3 instances of X”)

Examples

- Imperative / single
 - Docker
- Imperative / single or orchestration
 - Vagrant
- Declarative / on demand
 - Terraform, OpenTofu
- Declarative / continuous
 - Kubernetes (K8s)



Terraform

- Description: what must exist, and not how to build it
 - A folder of files with a .tf extension
- Provider: turns the description into API calls
 - Docker, Google Cloud, AWS, ...
 - Fixed to a known version
- Resource: one thing that must exist
 - A type, a name, and arguments
 - resource "docker_container" "hello" { ... }
- Reference: an argument that uses another resource
 - docker_image.hello.image_id
 - Order of operations is derived from these

Terraform workflow

- init: download the providers and record their versions
- plan: compare the description with reality, and show what would change
- apply: perform the changes
- destroy: remove everything that was created
- Applying twice changes nothing the second time
 - Same commands in Terraform and OpenTofu

Terraform state

- Three descriptions that can disagree:
 - Desired: what the files say
 - State: what was created, as recorded in a file
 - Reality: what exists right now
- plan compares them and proposes corrective action
 - Remove a container by hand, and plan proposes to create it again
- Only when it is run
 - No continuous reconciliation, unlike Kubernetes

Example with Docker and Terraform

- Install and run a Python / NumPy program
- Configuration:
 - Container limited to 1GB RAM and a CPU share
 - Ubuntu operating system
 - Install Python in the image, with PIP and Virtualenv
- Application setup:
 - Download requirements with PIP
 - Use a configuration variable
 - Run on startup



Example with Docker and Terraform

Select OS

```
FROM ubuntu:24.04
```

```
RUN apt-get update && \  
    apt-get install -y python3-pip python3-venv && \  
    rm -rf /var/lib/apt/lists/*
```

Install and configure
software

```
WORKDIR /app  
COPY hello.py .
```

```
RUN python3 -m venv venv && \  
    ./venv/bin/pip install numpy
```

Copy local files

Install user software



Example with Docker and Terraform

```
terraform {  
  required_providers {  
    docker = { source = "kreuzwerker/docker", version = "~> 4.6" }  
  }  
}  
variable "myparam" { default = 10 }  
  
resource "docker_image" "hello" {  
  build { context = "." }  
}  
resource "docker_container" "hello" {  
  image      = docker_image.hello.image_id  
  memory     = 1024  
  cpu_shares = 512  
  env        = ["MYPARAM=${var.myparam}"]  
  command    = ["sh", "-c", "hello.py $MYPARAM"]  
}
```

Pin the provider

Set configuration variables

Order comes from this reference

Limit resources

Run workload



Summary

- Virtualization technologies
 - Hypervisors virtualize hardware and containers virtualize the operating system
 - The cut is made at a different level of the stack
- Cloud services
 - IaaS, PaaS, and SaaS differ in what the provider operates
 - Managed elasticity from PaaS upwards
- Provisioning tools
 - Imperative describes the steps vs. declarative describes the result
 - Single instance, or orchestration of several
 - Infrastructure as code: a description kept and versioned as a program